HDOC Day-49

****

Write a C program that reads a positive integer n. Pass n to a function named pivot_integer() using its address. Use pointers to pass the value between the calling function and pivot_integer(). The function pivot_integer() must:

- Find the pivot integer x between 1 and n such that the sum of all integers from 1 to x, inclusively, is equal to the sum of all integers from x to n, inclusively.
- Display the pivot integer x if it exists.
- Display -1 if no such pivot integer exists.

1. Problem Understanding

The program reads a positive integer n and passes its address to pivot_integer(). The function checks each integer x from 1 to n and compares the inclusive sum from 1 to x with the inclusive sum from x to n. It returns the pivot integer when both sums are equal; otherwise, it returns -1.

2. Standard Test Cases

Test Case	Input (n)	Expected Output	Explanation
1	8	Pivot integer: 6	1+2+3+4+5+6 = 6+7+8
2	1	Pivot integer: 1	1 = 1
3	4	-1	No pivot integer exists
4	49	Pivot integer: 35	Inclusive sums are equal at x = 35
5	9	Pivot integer: 6	1+2+3+4+5+6 = 6+7+8+9

3. Algorithm

Main Algorithm

1. Start.
2. Read a positive integer n.
3. Pass the address of n to pivot_integer().
4. Store the value returned by the function.
5. Display the pivot integer if it exists; otherwise display -1.
6. Stop.

Function: pivot_integer()

1. Receive the address of n using an integer pointer.
2. For each x from 1 to n, calculate the sum of integers from 1 to x.
3. Calculate the sum of integers from x to n.
4. If both sums are equal, return x.
5. If no value of x satisfies the condition, return -1.

4. C Program

```
#include <stdio.h>

int pivot_integer(int *n)

{

   int x, i;

   int leftSum, rightSum;

   for(x = 1; x <= *n; x++)

   {

      leftSum = 0;

      rightSum = 0;

      for(i = 1; i <= x; i++)

      {

         leftSum += i;

      }

      for(i = x; i <= *n; i++)

      {

         rightSum += i;

      }

      if(leftSum == rightSum)

      {

         return x;

      }

   }

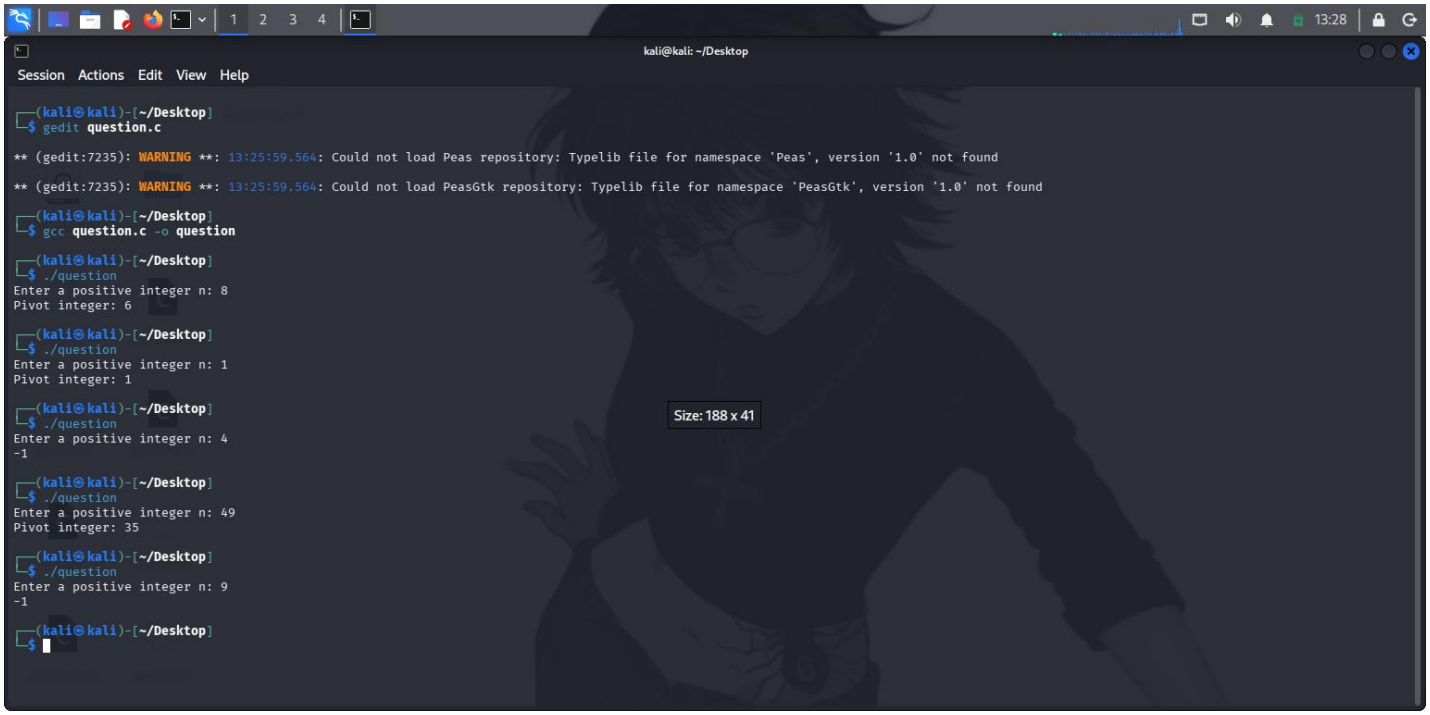
   return -1;

}

int main()
```

```
{
   int n, result;
   printf("Enter a positive integer n: ");
   scanf("%d", &n);
   result = pivot_integer(&n);
   if(result == -1)
      printf("-1\n");
   else
      printf("Pivot integer: %d\n", result);
  return 0;
}
```

5. Compilation and Execution Evidence



6. Verification of All Test Cases

Test Case	Input (n)	Actual Result	Expected Result	Status
1	8	Pivot integer: 6	Pivot integer: 6	Pass
2	1	Pivot integer: 1	Pivot integer: 1	Pass
3	4	-1	-1	Pass

4	49	Pivot integer: 35	Pivot integer: 35	Pass
5	9	Pivot integer: 6	Pivot integer: 6	Pass